

CIS 22 – INTRO TO PROGRAMMING

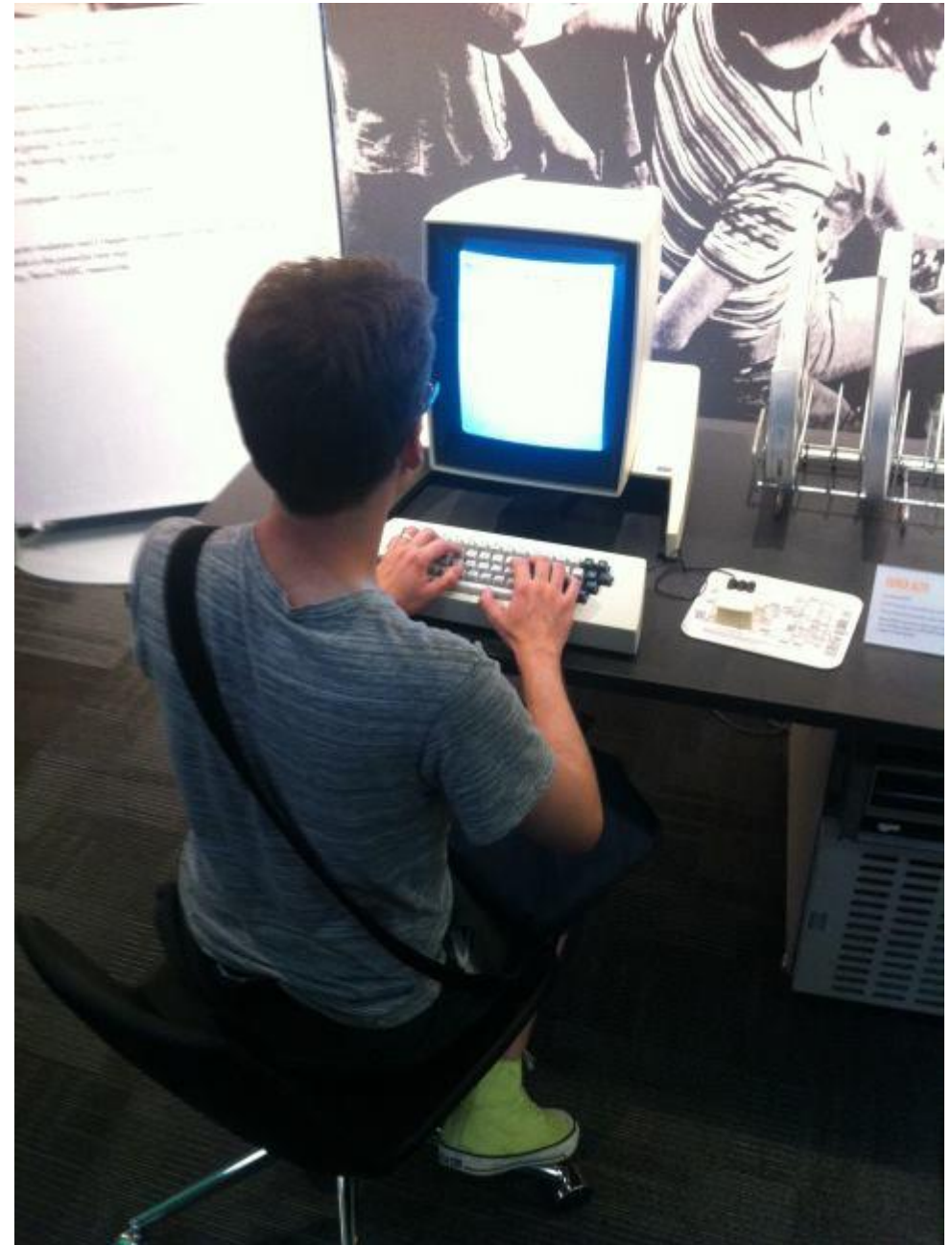
ABOUT THIS CLASS

practical details

- CIS 22: Intro to Programming
- NOTE: Despite the course name, This is NOT a good first programming class. There's going to be a bit of review at the front, but it'll ramp up real quick
- Meets Tuesdays and Thursdays in this room from 11:00 AM to 12:55 PM, typically with a 10 minute break toward the middle of the class.
- Although officially we have separate lab and lecture sections, generally Tuesday will be mostly lecture and Thursday mostly lab

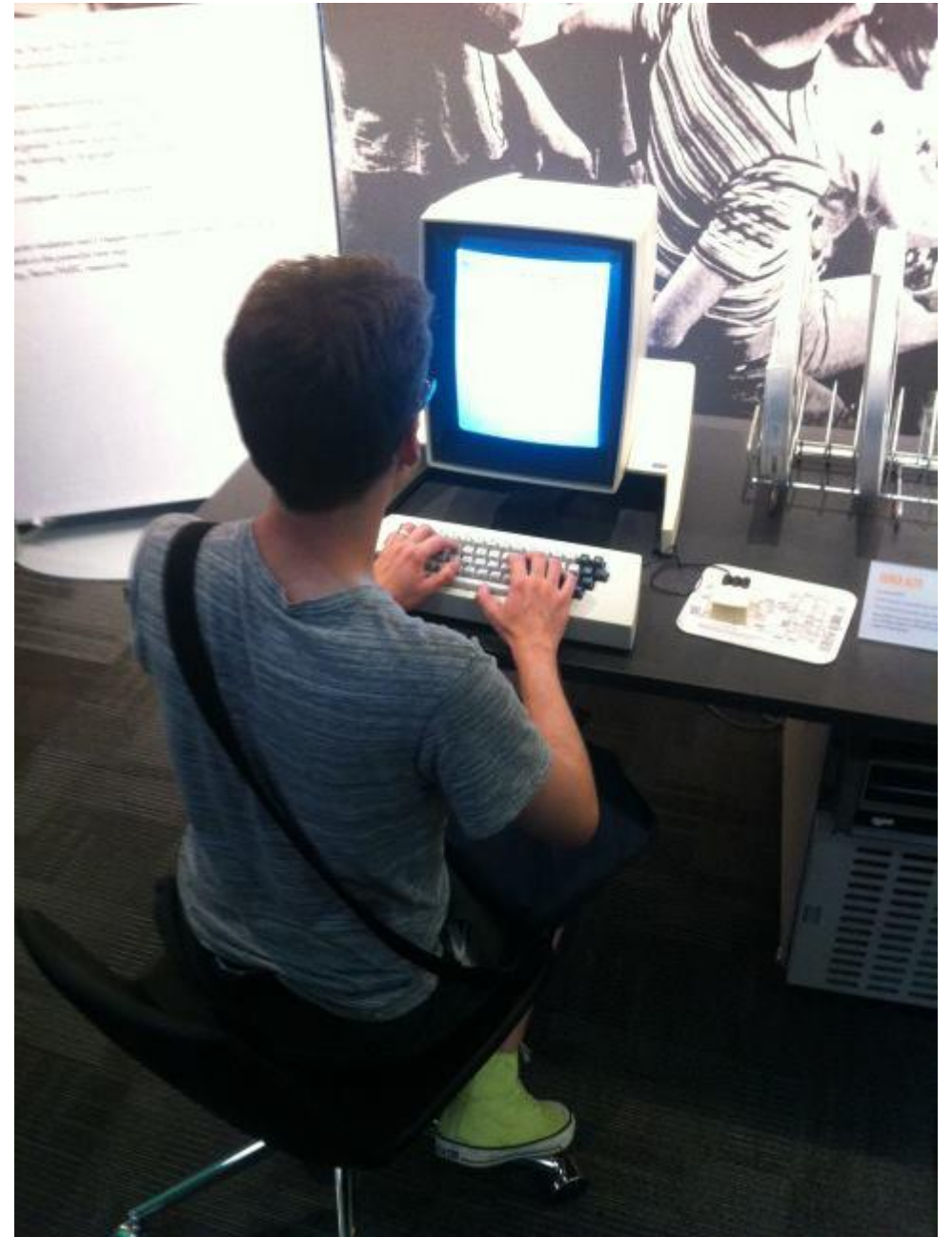
ABOUT ME

- Preferred pronouns: he/him
- My first degree was an associate's degree from Bellevue Community College
- A while ago I finished up a Master's in Computer Science at Stanford (I specialized in CS education and compiler design)



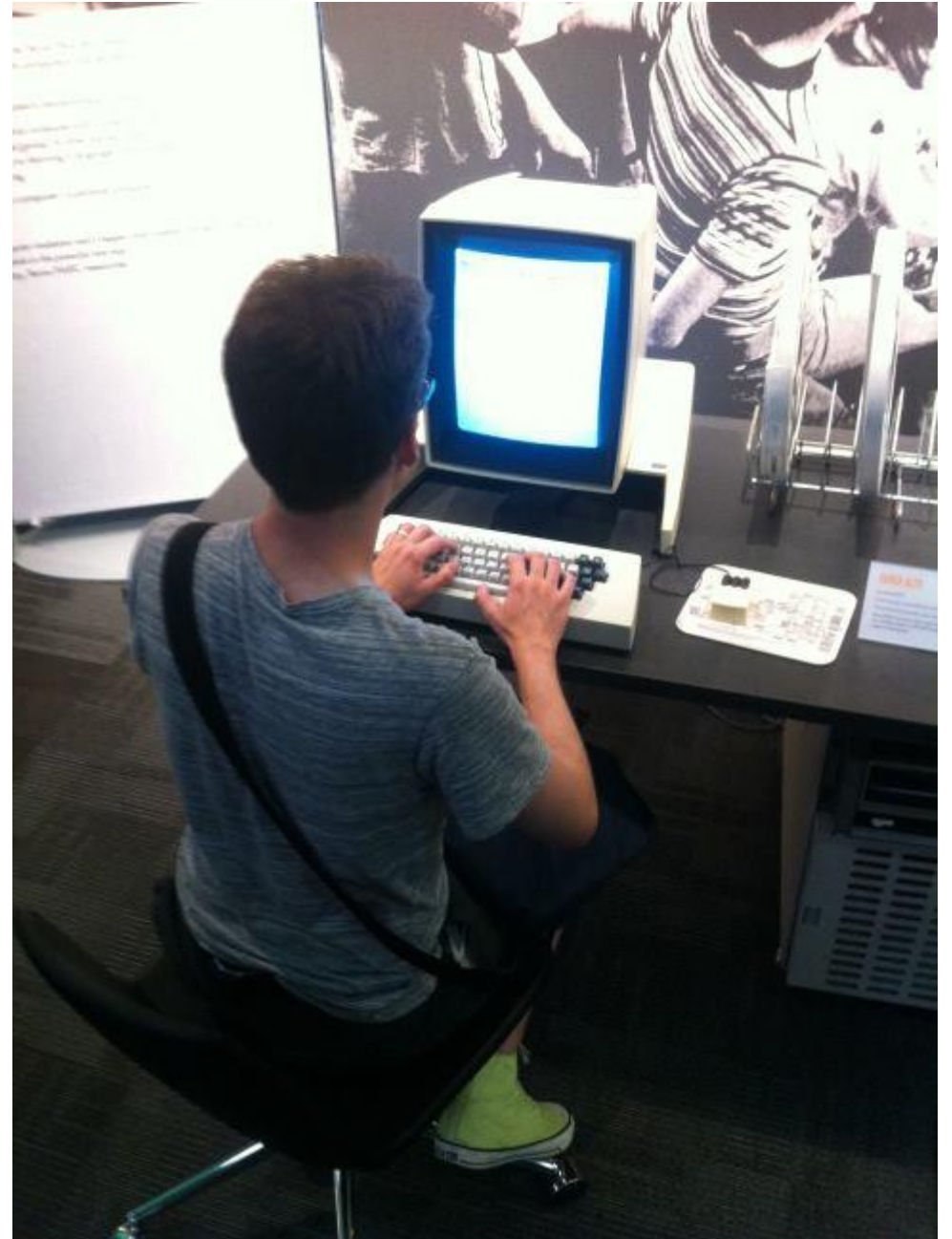
ABOUT ME

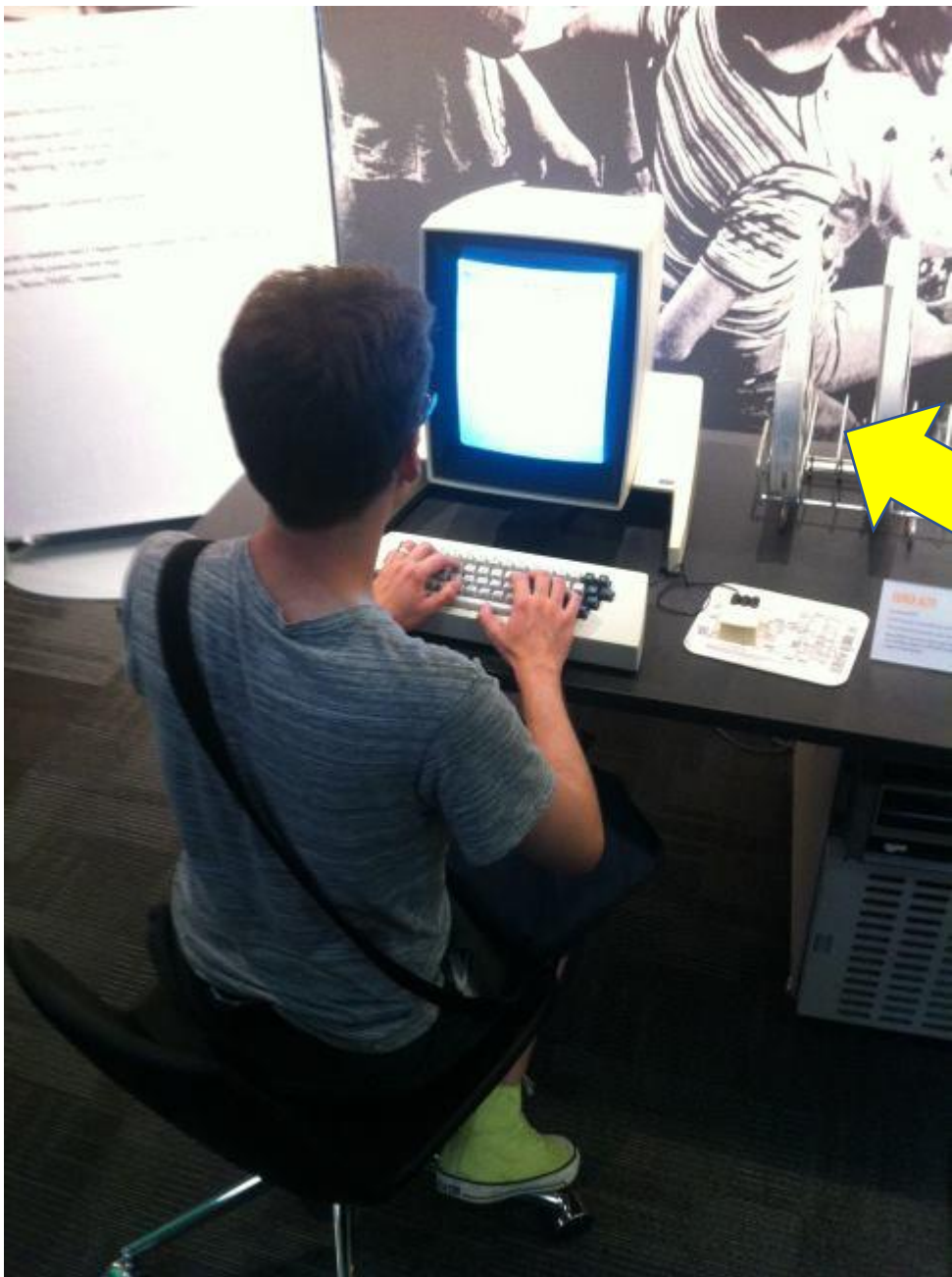
- Before that, I got a PhD in the history of computing (also from Stanford)
- (which is why I was so excited about using this computer)



ABOUT ME

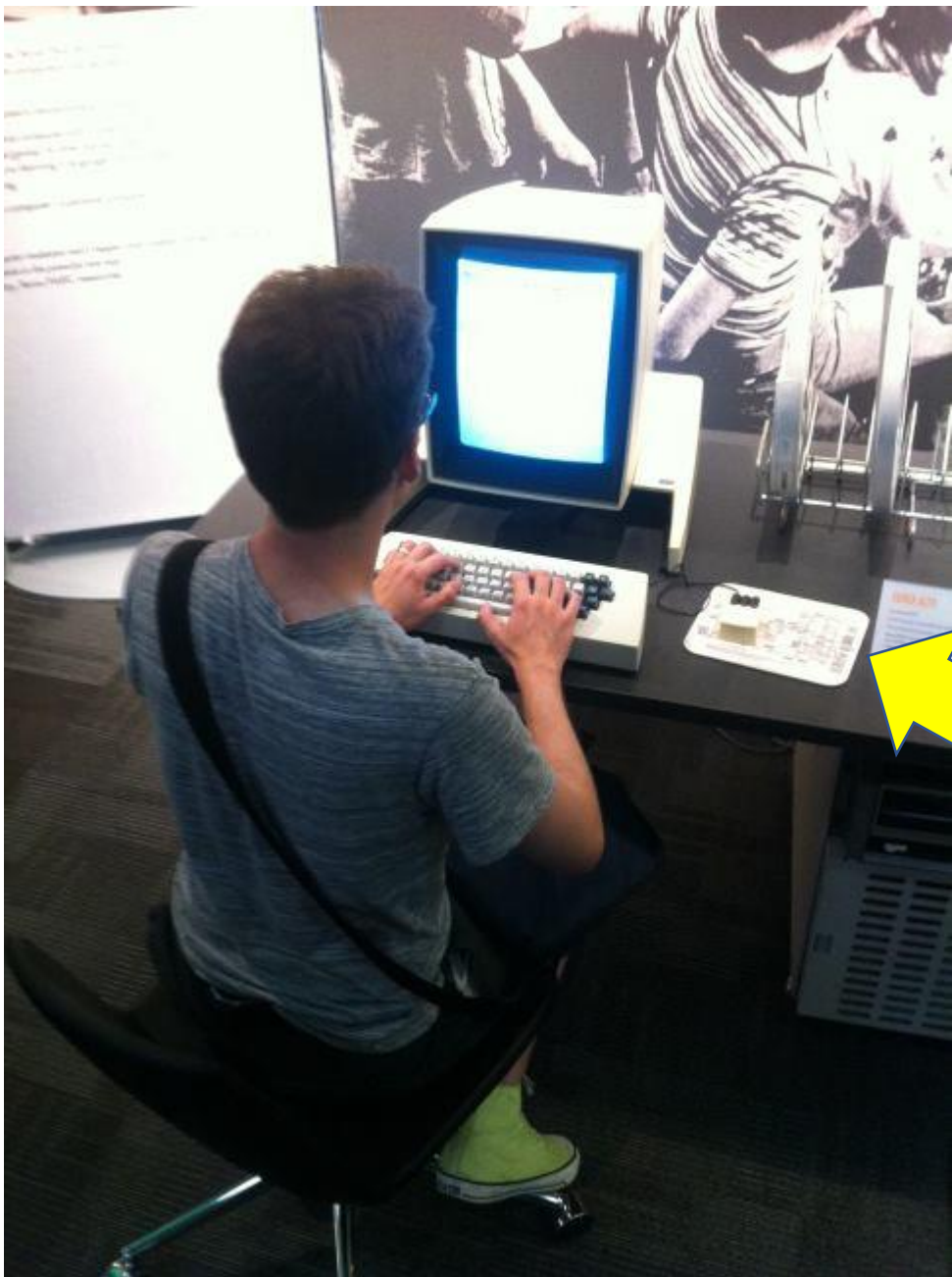
- (I'll sneak some material from the history of computing into class, but I promise to keep it under control)





ABOUT THIS PICTURE

- The machine I'm sitting at here is a Xerox Alto
- It was the first computer with a graphical user interface (windows, icons, pointer)
- First computer with a mouse!
- Built in 1973!



ABOUT THIS PICTURE

- The machine I'm sitting at here is a Xerox Alto
- It was the first computer with a graphical user interface (windows, icons, pointer)
- **First computer with a mouse!**
- Built in 1973!

ABOUT THIS CLASS: what you'll learn

- Here's what we all should know how to do by the end of the semester
- Design, implement, test, and debug programs that use conditional statements, loop structures, methods, and object-oriented concepts including classes, inheritance, arrays, virtual classes, templates, and exception handling for writing programs.

ABOUT THIS CLASS: What you should know coming in

You should have at least some experience with the following:

- Designing, implementing, testing, and debugging programs that use conditional statements, loop structures, and functions.
- Decomposing complex problems into relatively simple functions.
- Programs that use arrays.

ABOUT THIS CLASS: what you'll learn

- By the end of the semester, we should all be able to:
 - Design, implement, test, and debug programs that use classes, inheritance, arrays, virtual classes, templates, and exceptions.
 - Write programs that use pointers to implement the following data structures: linked lists, binary search trees, and heaps
 - Demonstrate an understanding of late binding and polymorphism.
 - Understand memory management in C++, including the distinction between stack-allocated memory and heap-allocated memory.
 - Demonstrate understanding of the importance of types, type incompatibility, and type-checking in providing abstraction and safety.
 - Produce implementations of breadth-first search, depth-first search, and other canonical computer science algorithms

ABOUT THIS CLASS: what you'll learn

- By the end of the semester, we should all be able to:
 - Design, implement, test, and debug programs that use classes, inheritance, arrays, virtual classes, templates, and exceptions.
 - Write programs that use pointers to implement the following data structures: linked lists, binary search trees, and heaps
 - Demonstrate an understanding of late binding and polymorphism.
 - Understand memory management in C++, including the distinction between stack-allocated memory and heap-allocated memory.
 - Demonstrate understanding of the importance of types, type incompatibility, and type-checking in providing abstraction and safety.
 - Produce implementations of breadth-first search, depth-first search, and other canonical computer science algorithms

This is the interesting stuff.
we're learning all this
syntax so that we can use it to
do real computer science.



ABOUT THIS CLASS: what you'll learn

- Demonstrate understanding of the importance of types, type incompatibility, and type-checking in providing abstraction and safety.



we're not just going to talk about how to do serious object-oriented programming in C++, but we're also going to get deep into the theory behind object oriented programming in general. basically you're going to understand how and why your compiler is helping you out when it gives you type errors.

... but before all that

- At the start of my CS classes, I like to do some exercises using a simple small language before we get to the large, complex languages we'll be working in for the rest of the semester.
- Why? We have to learn a lot of syntax to use a powerful language like C++, and sometimes that can obscure the fact that computer science is about *algorithm design* first and foremost.
- The exercises we'll do today will (hopefully) wake up the algorithm design parts of your brain a little bit.

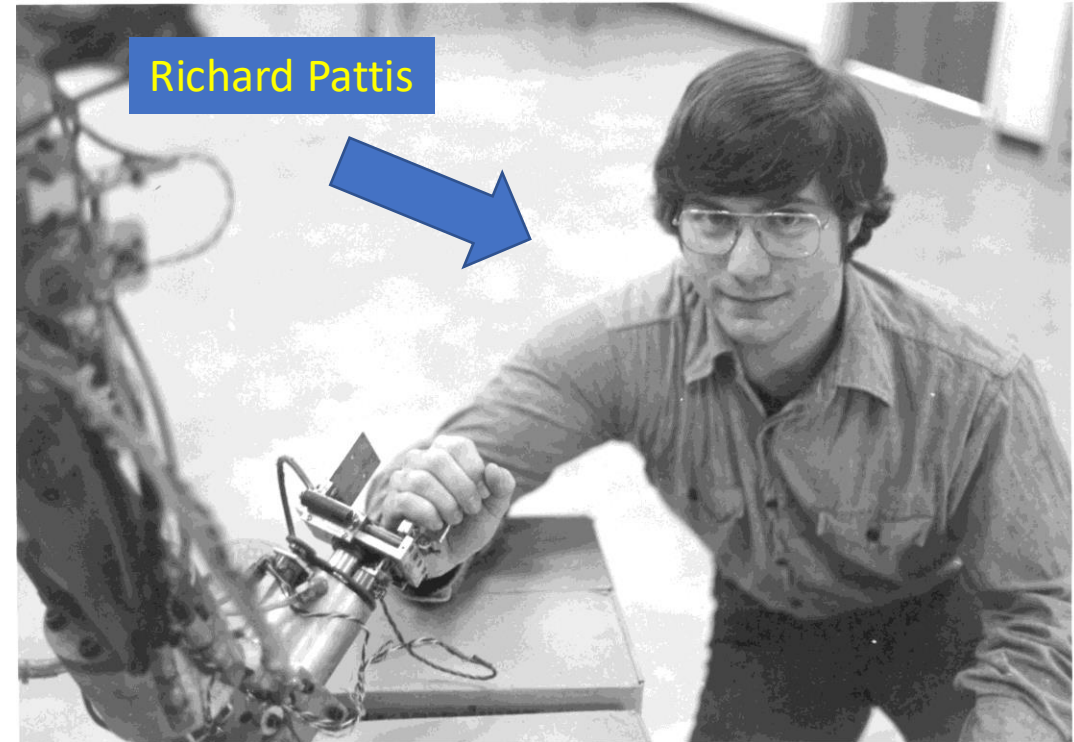


KAREL THE ROBOT

has escaped from Stanford

KAREL HISTORY

- Originally developed by a computer scientist named Richard Pattis while he was working on his master's degree at Stanford
- First released in 1981
- Karel has been used at Stanford ever since



KAREL HISTORY

- Originally developed by a computer scientist named Richard Pattis while he was working on his master's degree at Stanford
- First released in 1981
- Karel has been used at Stanford ever since
- This is only the second class at BCC to use Karel



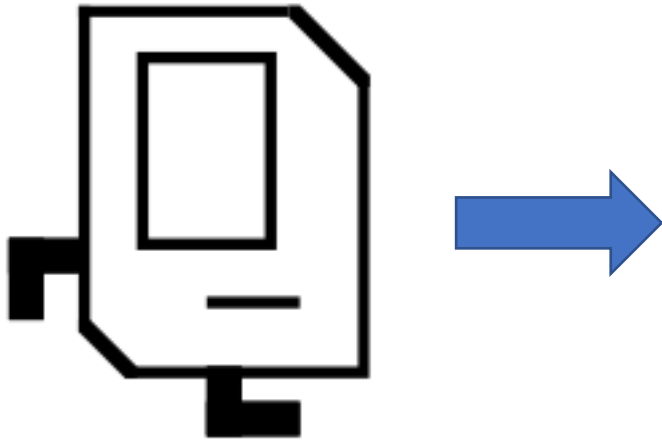
ABOUT KAREL

- Karel was born in 1981
- Karel looks a little bit like a 1980s Macintosh computer



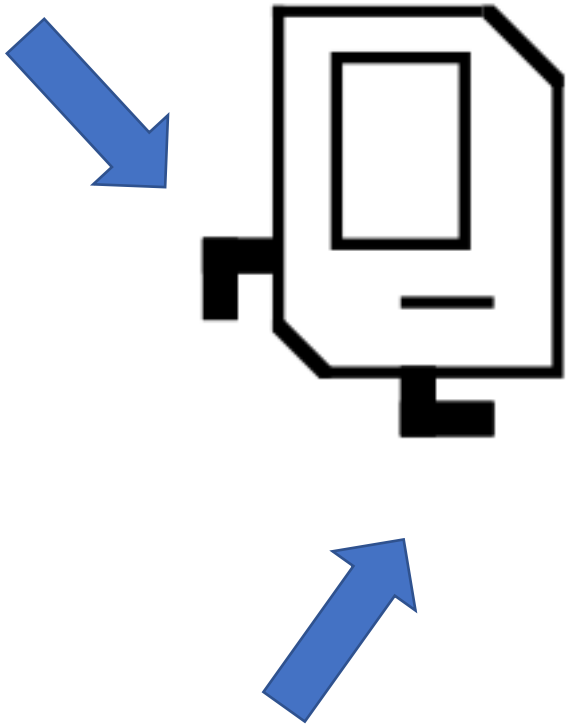
ABOUT KAREL

- Karel is facing RIGHT in this picture



ABOUT KAREL

- Karel is facing RIGHT in this picture
- These are Karel's feet

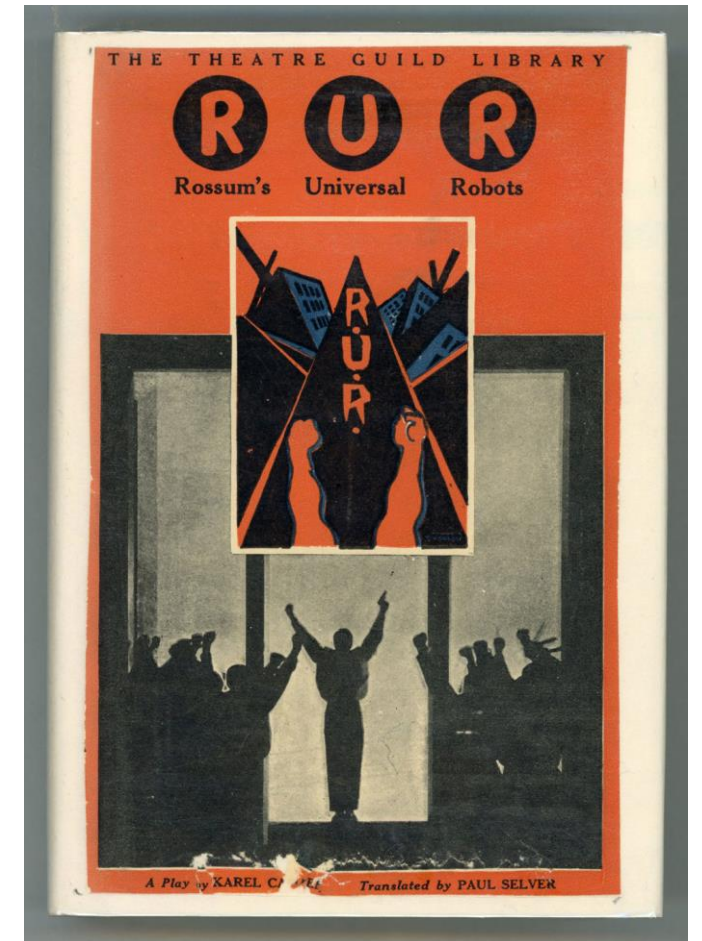


ABOUT KAREL



- Karel is named after Karel Čapek, the Czech playwright who invented the word "robot" back in 1920.

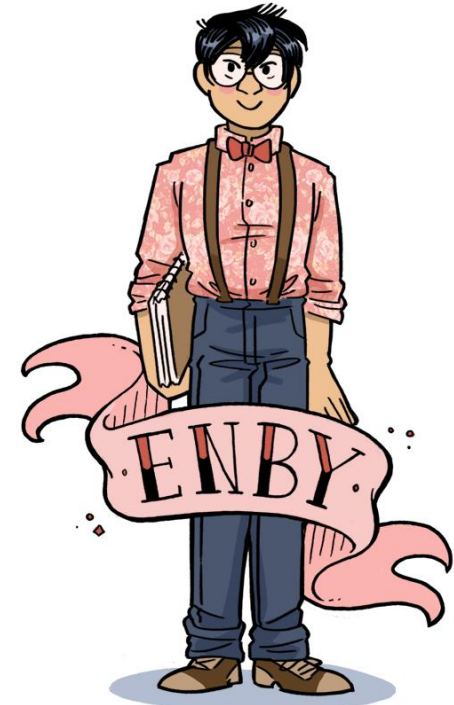
Rossum's Universal Robots. It's a pretty good play!



ABOUT KAREL



- Karel is a non-binary robot
- Karel uses she/her or they/their pronouns (either way is fine)



WHERE TO FIND KAREL



- We'll be using an online Karel IDE available here:
- <http://stanford.edu/~cpiech/karel/ide.html>
- Don't worry about the "learn" or "community" tabs – we'll JUST be using the IDE.

KAREL COMMANDS

- Here are Karel commands that definitely work:
 - `move();`
 - `turnLeft();`
 - `putBeeper();`
 - `pickBeeper();`



These are all FUNCTION CALLS. how do you know? the parentheses after each of them.

- If a command is by itself on a line, that's called a STATEMENT

```
move();  
turnLeft();  
putBeeper();
```



each of these are statements

- Single-line statements end with a ;

```
move();  
turnLeft();  
putBeeper();
```



If you forget the semicolon, your
program won't run and you will be sad 😞

- Karel also accepts a few COMPOUND STATEMENTS

```
while(frontIsClear()){  
}
```



this is a CONDITIONAL STATEMENT

this compound statement executes
only while this conditional statement is
TRUE



everything INSIDE the {

}

is part of the compound statement

(this is just like a while loop in C++, Java, or Python)

notice
that frontIsClear is a
FUNCTION – it's got parentheses after it



```
while(frontIsClear()){  
  
}
```



everything INSIDE the {

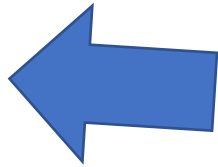
this is a CONDITIONAL STATEMENT

}

this compound statement executes
only while this conditional statement is
TRUE

is part of the compound statement

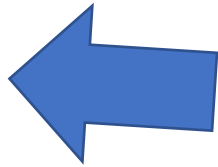
```
while(frontIsClear()){  
    move();  
    turnLeft();  
    move();  
}
```



notice that the COMPOUND STATEMENT
contains SIMPLE STATEMENTS

each of the simple statements ends with a
;

```
while(frontIsClear()){  
    move();  
    turnLeft();  
    move();  
}
```



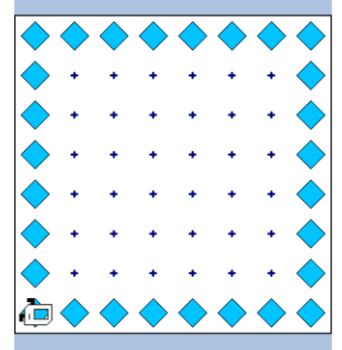
notice that the COMPOUND STATEMENT
contains SIMPLE STATEMENTS

each of the simple statements ends with a
;

If Karel doesn't have something stopping them from moving, they'll move forward, turn left, then move forward again. Then they'll check if their front is clear again. If it is, they repeat everything inside the { } block

SHORT ASSIGNMENT 1

- Go to <http://stanford.edu/~cpiech/karel/ide.html>

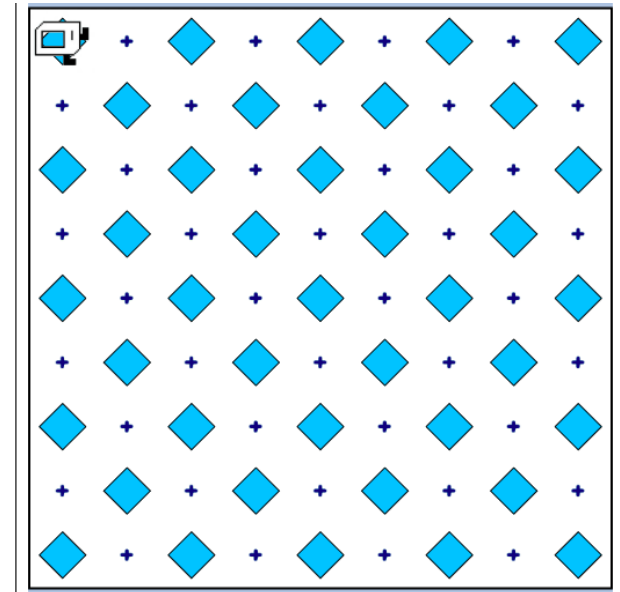


WARMUP:

- Make Karel put one beeper on every square at the edge of the world, then reverse direction and pick up the beeper. You must use at least one function other than `main()`, and at least one while loop

THE FUN STUFF:

- Make Karel draw a *checkerboard* pattern on the 9x9 world.



TURN THIS IN WHEN DONE. then move on to...